SecureQT SDK (`secureqt-sdk`)

A Kotlin Multiplatform library for **end-to-end encrypted data transfer over a physical USB cable** using the **Android Open Accessory (AOA)** protocol.

The SDK ships as two binaries:

Artifact	Platform	Role	Use it in
`secureqt-sdk-jvm.jar`	JVM (Java 11+)	**USB Host / Initiator**	Desktop apps (Windows / Linux / macOS)
`secureqt-sdk-android.aar`	Android (minSdk 24)	**USB Accessory / Responder**	Android apps

- **Version:** `1.0.0` (`SecureQtSdk.VERSION`)
- **Package:** `com.example.secureqt.sdk`
- **Crypto:** ECDH P-256 ephemeral key exchange → AES-256-GCM for every payload

---

Table of Contents

1. How it works
2. Getting the binaries
3. Desktop (JAR) integration
4. Android (AAR) integration
5. Standard commands & wire contract
6. Custom commands
7. API reference
8. Limits, threading & gotchas
9. Troubleshooting

---

1. How it works

```
Desktop (JAR) — USB Host / Initiator          Android (AAR) — Accessory / Responder
──────────────────────────────────────        ──────────────────────────────────────

1. switchDeviceToAoa()
     AOA control requests 51/52/53 ─────────► device re-enumerates as accessory
                                              USB_ACCESSORY_ATTACHED intent fires
2. re-scan bus, open, claim interface
3. createDesktopTransport()                   createAndroidAccessoryTransport()
4. performHandshake(isInitiator = true) ◄─► performHandshake(isInitiator = false)
          (ECDH P-256 → shared AES-256-GCM key)
5. createTransferClient()
     sendFile / fetchFile / ... ──────────────► receiveSecure() → parse → handle
                                ◄────────────── reply via sendSecure()
```

Layers (all in `commonMain`, shared by both binaries):

Layer	Class	Responsibility
Transport	`IUsbTransport`	Raw byte read/write over USB
Framing	`PacketCodec`, `PacketTypes`	`[1: type][4: length][payload]` packets

Layer	Class	Responsibility
Security	`SecureChannel`	ECDH handshake, AES-GCM encrypt/decrypt, framing
Commands	`CommandPayloadBuilder` / `CommandPayloadParser`	`[1: commandId][args…]` payloads
High-level client	`ISecureTransferClient`	File/folder send, fetch, list, delete, rename… (host side)

> **Important:** the high-level `ISecureTransferClient` is the **host side only**. The Android side gets the transport + channel + parser. **Your Android app implements the command handlers** (see §4.5).

---

2. Getting the binaries

Option A: Download from GitHub Releases

Pushing a tag matching `sdk-v*` (for example `sdk-v1.0.0`) runs `.github/workflows/sdk-release.yml`, which attaches these files to the release:

- `secureqt-sdk-jvm.jar`
- `secureqt-sdk-android.aar`
- `usb-driver-installer-standalone.jar` (Windows WinUSB driver helper)

Option B: Build locally

From the repository root:

```
# JVM JAR → secureqt-sdk/build/libs/secureqt-sdk-jvm.jar
./gradlew :secureqt-sdk:jvmJar

# Android AAR → secureqt-sdk/build/outputs/aar/secureqt-sdk-release.aar
./gradlew :secureqt-sdk:bundleReleaseAar
```

Requires JDK 17 to run Gradle (the SDK compiles to JVM 11 bytecode) and Android SDK platform 36.

Option C: Same Gradle build (module dependency)

```
// settings.gradle.kts
include(":secureqt-sdk")

// consumer build.gradle.kts
dependencies { implementation(project(":secureqt-sdk")) }
```

---

3. Desktop (JAR) integration

3.1 Add dependencies

The JAR is a **plain (thin) JAR**. It does not include its dependencies, so you must add them yourself:

```
// build.gradle.kts (Desktop / JVM project)
dependencies {
    implementation(files("libs/secureqt-sdk-jvm.jar"))

    // Required transitive dependencies
    implementation("org.usb4java:usb4java:1.3.0")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.10.2")
}
```

Maven

```
<dependency>
  <groupId>com.example.secureqt</groupId>
  <artifactId>secureqt-sdk-jvm</artifactId>
  <version>1.0.0</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/libs/secureqt-sdk-jvm.jar</systemPath>
</dependency>
<dependency>
  <groupId>org.usb4java</groupId>
  <artifactId>usb4java</artifactId>
  <version>1.3.0</version>
</dependency>
<dependency>
  <groupId>org.jetbrains.kotlinx</groupId>
  <artifactId>kotlinx-coroutines-core-jvm</artifactId>
  <version>1.10.2</version>
</dependency>
```

3.2 OS prerequisites

OS	Requirement
Windows	The AOA device (VID `0x18D1`, PID `0x2D00` / `0x2D01`) needs a **WinUSB** driver, and `adb` must not hold the device. See §3.7.
Linux	A udev rule so non-root users can open the phone and the AOA device. See §3.8.
macOS	Works with the bundled libusb from usb4java; no driver needed.

3.3 Minimal host flow (quick start)

The example below shows the bare sequence. For a production host that handles the Windows and Linux differences (driver binding, `adb`, udev permissions, retries), use `SecureQtHost` from §3.5.

```
import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.createDesktopTransport
import com.example.secureqt.sdk.findDesktopBulkEndpoints
import com.example.secureqt.sdk.switchDeviceToAoa
import kotlinx.coroutines.runBlocking
import org.usb4java.*
import java.io.File

const val AOA_VID: Short = 0x18D1
val AOA_PIDS = 0x2D00..0x2D05

fun findDevice(predicate: (DeviceDescriptor) -> Boolean): Device? {
    val list = DeviceList()
    LibUsb.getDeviceList(null, list)
    try {
        for (dev in list) {
            val desc = DeviceDescriptor()
            LibUsb.getDeviceDescriptor(dev, desc)
            if (predicate(desc)) return LibUsb.refDevice(dev)
        }
    } finally {
        LibUsb.freeDeviceList(list, true)
    }
```

```
    return null
}

fun main() = runBlocking {
    check(LibUsb.init(null) == LibUsb.SUCCESS)

    // ── 1. Find the phone and switch it into AOA mode (skip if already AOA) ──
    var aoa = findDevice { it.idVendor() == AOA_VID && (it.idProduct().toInt() and 0xFFFF) in AOA_PIDS }
    if (aoa == null) {
        val phone = findDevice { it.idVendor() == 0x04E8.toShort() } // your phone's normal VID
            ?: error("No Android device found")
        // manufacturer/model MUST match the Android app's accessory_filter.xml
        SecureQtSdk.switchDeviceToAoa(phone, manufacturer = "AndroidOpenAccessory", model = "DataTransfer
")

        // ── 2. Device disconnects and re-enumerates as 18D1:2D0x; poll for it ──
        repeat(20) {
            Thread.sleep(500)
            aoa = findDevice { it.idVendor() == AOA_VID && (it.idProduct().toInt() and 0xFFFF) in AOA_PID
S }
            if (aoa != null) return@repeat
        }
    }
    val device = aoa ?: error("Device did not re-enumerate in AOA mode")

    // ── 3. Open + claim the interface (the SDK does NOT claim it for you) ──
    val handle = DeviceHandle()
    check(LibUsb.open(device, handle) == LibUsb.SUCCESS)
    val ep = SecureQtSdk.findDesktopBulkEndpoints(device) ?: error("No bulk endpoints")
    LibUsb.setAutoDetachKernelDriver(handle, true)
    check(LibUsb.claimInterface(handle, ep.interfaceNumber) == LibUsb.SUCCESS)

    val transport = SecureQtSdk.createDesktopTransport(
        handle = handle,
        inEp = ep.inAddr,
        outEp = ep.outAddr,
        ifaceNum = ep.interfaceNumber
    )

    // ── 4. Encrypted channel + ECDH handshake (host is the initiator) ──
    val channel = SecureQtSdk.createChannel(transport)
    if (!channel.performHandshake(isInitiator = true)) error("Handshake failed")

    // ── 5. High-level operations ──
    val client = SecureQtSdk.createTransferClient(channel)

    client.listRemoteFiles("/storage/emulated/0/Download").forEach {
        println("${if (it.isDirectory) "[D]" else "   "} ${it.name}  ${SecureQtSdk.Utils.formatSize(it.si
ze)}")
    }

    client.sendFile(File("photo.jpg"), "/storage/emulated/0/Download") { sent, total ->
        println("Upload ${sent * 100 / total}%")
    }

    client.fetchFile("/storage/emulated/0/Download/report.pdf", File("downloads"))
    client.sendDirectory(File("myFolder"), "/storage/emulated/0/Download")     // streamed as ZIP
    client.fetchRemoteDirectory("/storage/emulated/0/DCIM/Camera", File("backup"))
    client.createFolder("/storage/emulated/0/Download/NewFolder")
    client.renameFile("/storage/emulated/0/Download/a.txt", "b.txt")
    client.deleteFile("/storage/emulated/0/Download/b.txt")

    // ── 6. Tear down ──
    client.sendDisconnect()
    transport.close()   // releases interface, re-attaches kernel driver, closes handle
    LibUsb.unrefDevice(device)
    LibUsb.exit(null)
}
```

3.4 Cancelling a transfer

```
val job = scope.launch { client.sendFile(bigFile, "/storage/emulated/0/Download") }
// later, e.g. from a "Cancel" button:
client.cancelTransfer()   // sends TYPE_CANCEL; the in-flight call returns false
```

3.5 Production host for Windows & Linux

The SDK gives you the AOA switch, endpoint discovery and the transport. Making the connection reliable on a real PC needs OS-specific handling, which is what this host layer adds:

Concern	Windows	Linux
USB access	libusb needs a **WinUSB** (or libusbK) driver bound to the device	libusb uses `usbfs`; the user needs **read/write permission** on `/dev/bus/usb/BBB/DDD` (udev rule)
Opening the phone **before** the AOA switch	Needs at least one interface with a WinUSB-compatible driver (for example the ADB interface with USB debugging on). Otherwise install WinUSB for the phone's own VID:PID	Works for any user the udev rule allows
Opening the AOA device (`18D1:2D00/2D01`)	Needs WinUSB on that VID:PID. On `2D01` (AOA + ADB composite) it must be on interface 0 (`MI_00`)	Works once permissions apply
Typical `LibUsb.open` errors	`-3 ERROR_ACCESS` (`adb.exe` holds the device), `-12 ERROR_NOT_SUPPORTED` (no WinUSB, or Plug and Play binding not finished)	`-3 ERROR_ACCESS` (no udev rule, or udev hasn't applied it yet just after re-enumeration)
Kernel driver on the interface	Not applicable (`setAutoDetachKernelDriver` isn't supported)	Detach it before claiming (`setAutoDetachKernelDriver(true)`)
Time to re-appear after the AOA switch	1–3 s, plus driver binding	~0.5–1.5 s
Admin rights	Only for the one-time driver install (UAC prompt)	Only for the one-time udev rule install

`HostPlatform`: per-OS behaviour

```
package com.yourcompany.secureqt.host

import com.example.usbdriverinstaller.api.IUsbDriverInstaller
import com.example.usbdriverinstaller.api.UsbDriverInstallerFactory
import org.usb4java.LibUsb
import java.io.File
import java.util.concurrent.TimeUnit

/** OS-specific policy used by [SecureQtHost]. */
interface HostPlatform {
    /** Extra wait after the AOA device appears, so the OS can finish binding drivers. */
    val settleDelayMs: Long

    /** Called once at the start of connect(), before the AOA switch. */
    fun prepare() {}

    /**
     * Called when `switchDeviceToAoa` fails on the phone in normal mode.
     * @return true to retry the switch.
     */
    fun onSwitchFailed(vid: Int, pid: Int, attempt: Int): Boolean = false

    /**
```

```
     * Called when opening/claiming the AOA device fails with libusb [error].
     * @return true to retry (the host re-scans the bus before each retry).
     */
    fun onOpenFailed(vid: Int, pid: Int, error: Int, attempt: Int): Boolean

    companion object {
        fun forCurrentOs(): HostPlatform {
            val os = System.getProperty("os.name", "").lowercase()
            return when {
                "win" in os -> WindowsHostPlatform()
                "linux" in os -> LinuxHostPlatform()
                else -> DefaultHostPlatform()     // macOS: no driver or permission work needed
            }
        }
    }
}

class DefaultHostPlatform : HostPlatform {
    override val settleDelayMs = 300L
    override fun onOpenFailed(vid: Int, pid: Int, error: Int, attempt: Int) = attempt < 5
}
```

Windows implementation

```
class WindowsHostPlatform(
    /** WinUSB installer from `usb-driver-installer-standalone.jar`; null = never install drivers. */
    private val installer: IUsbDriverInstaller? =
        runCatching { UsbDriverInstallerFactory.getDefaultInstaller() }.getOrNull(),
    /** Set false if your users must keep MTP (Explorer access) on the phone's normal VID:PID. */
    private val allowDriverInstallOnPhone: Boolean = true
) : HostPlatform {

    override val settleDelayMs = 800L          // Plug and Play needs time to bind the new device node

    /** adb.exe opens every Android device it sees and blocks libusb with ERROR_ACCESS. */
    override fun prepare() = killAdbServer()

    override fun onSwitchFailed(vid: Int, pid: Int, attempt: Int): Boolean {
        // The phone has no WinUSB-compatible interface (USB debugging off, or no ADB driver).
        if (attempt > 1 || !allowDriverInstallOnPhone) return false
        return installWinUsb(vid, pid, "Android Device")
    }

    override fun onOpenFailed(vid: Int, pid: Int, error: Int, attempt: Int): Boolean = when (error) {
        LibUsb.ERROR_ACCESS, LibUsb.ERROR_BUSY -> {
            killAdbServer()
            attempt < 16
        }
        LibUsb.ERROR_NOT_SUPPORTED, LibUsb.ERROR_NOT_FOUND -> {
            // Retry the first few times: the driver may be installed but not bound yet.
            // Only after that, install WinUSB for the AOA VID:PID.
            if (attempt == 4) installWinUsb(vid, pid, "Android Open Accessory") else attempt < 16
        }
        else -> attempt < 3
    }

    private fun installWinUsb(vid: Int, pid: Int, name: String): Boolean {
        val inst = installer ?: return false
        if (inst.isDriverAlreadyInstalled(vid, pid)) return true
        // One UAC prompt. installWinUsb() also removes conflicting OEM drivers and
        // rescans/restarts the device, so don't call cleanupConflictingDrivers()/
rescanAndRestartDevice() too.
        val result = inst.installWinUsb(vid, pid, name)
        if (!result.isSuccess) {
            System.err.println("[SecureQT] WinUSB install failed for %04X:%04X: $
{result.message}".format(vid, pid))
            return false
        }
        Thread.sleep(1500)                                          // let Plug and Play bind the driver
        return true
    }

    private fun killAdbServer() {
        runCatching {
```

```
            val sdkAdb = System.getenv("LOCALAPPDATA")
                ?.let { File(it, "Android\\Sdk\\platform-tools\\adb.exe") }
                ?.takeIf { it.exists() }
            val cmd = if (sdkAdb != null) listOf(sdkAdb.absolutePath, "kill-server")
                      else listOf("taskkill", "/F", "/IM", "adb.exe")
            ProcessBuilder(cmd).redirectErrorStream(true).start().waitFor(3, TimeUnit.SECONDS)
        }
    }
}
```

Linux implementation

```
class LinuxHostPlatform : HostPlatform {

    override val settleDelayMs = 300L

    override fun onOpenFailed(vid: Int, pid: Int, error: Int, attempt: Int): Boolean {
        // Right after re-enumeration, udev may not have applied the rule yet, so retry briefly.
        if (attempt < 6) return true
        if (error == LibUsb.ERROR_ACCESS) {
            System.err.println(
                "[SecureQT] Permission denied on %04x:%04x. Install the udev rule (README §3.8),
".format(vid, pid) +
                "then unplug and replug the phone."
            )
        }
        return false
    }
}
```

`SecureQtHost` : the cross-platform host

```
package com.yourcompany.secureqt.host

import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.client.ISecureTransferClient
import com.example.secureqt.sdk.createDesktopTransport
import com.example.secureqt.sdk.findDesktopBulkEndpoints
import com.example.secureqt.sdk.switchDeviceToAoa
import com.example.secureqt.sdk.transport.desktop.DesktopAoaTransport
import org.usb4java.Context
import org.usb4java.Device
import org.usb4java.DeviceDescriptor
import org.usb4java.DeviceHandle
import org.usb4java.DeviceList
import org.usb4java.LibUsb

/**
 * Finds an Android phone, switches it to AOA mode, opens it with OS-specific recovery,
 * and performs the SecureQT handshake. One instance = one phone connection.
 */
class SecureQtHost(
    private val platform: HostPlatform = HostPlatform.forCurrentOs(),
    /** Must match the Android app's res/xml/accessory_filter.xml. */
    private val manufacturer: String = "AndroidOpenAccessory",
    private val model: String = "DataTransfer",
    /** Restrict to one phone, e.g. 0x04E8 (Samsung). null = any known Android vendor. */
    private val targetVid: Int? = null
) : AutoCloseable {

    companion object {
        const val AOA_VID = 0x18D1
        val AOA_PIDS = 0x2D00..0x2D05
        val ANDROID_VIDS = setOf(
            0x18D1, 0x04E8, 0x2717, 0x2A70, 0x22D9, 0x2B4C, 0x1EBF, 0x22B8, 0x0FCE, 0x1004,
            0x0BB4, 0x19D2, 0x05C6, 0x0E8D, 0x12D1, 0x0B05, 0x17EF, 0x29D1, 0x3170, 0x1949
        )
        private const val MAX_OPEN_ATTEMPTS = 20
    }

    private val ctx = Context()
    private var transport: DesktopAoaTransport? = null
```

```
    var client: ISecureTransferClient? = null
        private set

    init {
        val r = LibUsb.init(ctx)
        check(r == LibUsb.SUCCESS) { "LibUsb.init failed: ${LibUsb.strError(r)}" }
    }

    /** Full connect: switch → re-enumerate → open/claim → ECDH handshake. */
    suspend fun connect(): ISecureTransferClient {
        platform.prepare()

        // 1. Switch to AOA unless the phone is already in accessory mode
        if (findDevice(isAoa) == null) {
            val phone = findDevice { vid, pid -> !isAoa(vid, pid) && (targetVid?.let { it == vid } ?: (vi
d in ANDROID_VIDS)) }
                ?: error("No Android device found. Check the cable and that the phone is unlocked.")
            try {
                val (vid, pid) = ids(phone)
                var attempt = 1
                while (!SecureQtSdk.switchDeviceToAoa(phone, manufacturer, model)) {
                    if (!platform.onSwitchFailed(vid, pid, attempt++)) {
                        error("AOA switch failed for %04X:%04X (driver/permission problem, see README
§3.7 / §3.8)".format(vid, pid))
                    }
                }
            } finally {
                LibUsb.unrefDevice(phone)
            }

            // 2. Phone disconnects and re-enumerates as 18D1:2D0x. Wait up to 10 s
            val deadline = System.currentTimeMillis() + 10_000
            while (findDevice(isAoa)?.also { LibUsb.unrefDevice(it) } == null) {
                if (System.currentTimeMillis() > deadline) {
                    error("Phone did not re-appear in AOA mode. Unlock it, accept the accessory prompt,
disable USB tethering.")
                }
                Thread.sleep(200)
            }
        }
        Thread.sleep(platform.settleDelayMs)

        // 3. Open + claim with OS-specific recovery
        val t = openAccessory()
        transport = t

        // 4. Encrypted channel (host = initiator)
        val channel = SecureQtSdk.createChannel(t)
        if (!channel.performHandshake(isInitiator = true)) {
            close()
            error("Handshake failed: the Android app didn't respond within ~30 s")
        }
        return SecureQtSdk.createTransferClient(channel).also { client = it }
    }

    private fun openAccessory(): DesktopAoaTransport {
        var lastError = LibUsb.ERROR_NO_DEVICE
        for (attempt in 1..MAX_OPEN_ATTEMPTS) {
            val dev = findDevice(isAoa)
            if (dev == null) { Thread.sleep(500); continue }
            val (vid, pid) = ids(dev)
            try {
                val handle = DeviceHandle()
                lastError = LibUsb.open(dev, handle)
                if (lastError == LibUsb.SUCCESS) {
                    val ep = SecureQtSdk.findDesktopBulkEndpoints(dev)
                    if (ep == null) {
                        lastError = LibUsb.ERROR_NOT_FOUND
                    } else {
                        // Linux: unbind any kernel driver. Windows: not supported, skip.
                        if (LibUsb.hasCapability(LibUsb.CAP_SUPPORTS_DETACH_KERNEL_DRIVER)) {
                            LibUsb.setAutoDetachKernelDriver(handle, true)
                        }
                        lastError = LibUsb.claimInterface(handle, ep.interfaceNumber)
                        if (lastError == LibUsb.SUCCESS) {
                            return SecureQtSdk.createDesktopTransport(handle, ep.inAddr, ep.outAddr, ep.i
nterfaceNumber)
```

```
                        }
                    }
                    LibUsb.close(handle)
                }
            } finally {
                LibUsb.unrefDevice(dev)
            }
            if (!platform.onOpenFailed(vid, pid, lastError, attempt)) break
            Thread.sleep(500)
        }
        error("Cannot open AOA device: ${LibUsb.strError(lastError)} ($lastError)")
    }

    /** Graceful disconnect: tell the phone, then release USB. */
    suspend fun disconnect() {
        runCatching { client?.sendDisconnect() }
        close()
    }

    override fun close() {
        transport?.close()        // releases interface, re-attaches kernel driver (Linux), closes handle
        transport = null
        client = null
    }

    /** Call once when the app exits. */
    fun shutdown() {
        close()
        LibUsb.exit(ctx)
    }

    // ── helpers ──
    private val isAoa: (Int, Int) -> Boolean = { vid, pid -> vid == AOA_VID && pid in AOA_PIDS }

    private fun ids(device: Device): Pair<Int, Int> {
        val d = DeviceDescriptor()
        LibUsb.getDeviceDescriptor(device, d)
        return (d.idVendor().toInt() and 0xFFFF) to (d.idProduct().toInt() and 0xFFFF)
    }

    /** Returns a referenced device (caller must unrefDevice) or null. */
    private fun findDevice(match: (vid: Int, pid: Int) -> Boolean): Device? {
        val list = DeviceList()
        if (LibUsb.getDeviceList(ctx, list) < 0) return null
        try {
            for (dev in list) {
                val (vid, pid) = ids(dev)
                if (match(vid, pid)) return LibUsb.refDevice(dev)
            }
        } finally {
            LibUsb.freeDeviceList(list, true)
        }
        return null
    }
}
```

Using it (same code on Windows and Linux)

```
import kotlinx.coroutines.runBlocking
import java.io.File

fun main() = runBlocking {
    val host = SecureQtHost()                    // picks WindowsHostPlatform / LinuxHostPlatform
automatically
    try {
        val client = host.connect()
        client.sendDeviceId("Workstation-01")
        client.sendFile(File("firmware.bin"), "/storage/emulated/0/Download") { sent, total ->
            print("\rUploading ${sent * 100 / maxOf(total, 1)}%")
        }
        println()
        client.listRemoteFiles("/storage/emulated/0/Download").forEach { println(it.name) }
        host.disconnect()
    } finally {
```

```
        host.shutdown()
    }
}
```

> **Threading:** `connect()` calls `Thread.sleep` while it waits for the device to re-enumerate. In a UI app, call it from `withContext(Dispatchers.IO) { host.connect() }`.

3.6 Host dependencies per OS

```
dependencies {
    implementation(files("libs/secureqt-sdk-jvm.jar"))
    implementation("org.usb4java:usb4java:1.3.0")   // includes x86-64 libusb natives
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.10.2")

    // Windows only (needed by WindowsHostPlatform for automatic WinUSB install).
    // Safe to ship on Linux too: the installer reports NotWindows there.
    implementation(files("libs/usb-driver-installer-standalone.jar"))
}
```

If you don't ship the driver installer, pass `WindowsHostPlatform(installer = null)`. Users then install WinUSB once with Zadig (§3.7).

> The driver installer has its own guide: **`usb-driver-installer/README.md`** (PDF: `usb-driver-installer/USB_Driver_Installer_Guide.pdf`). It covers the full API, the managed state machine for UI dialogs, what an install changes on the PC, and security considerations for corporate machines.

3.7 Windows setup

One-time per PC (per phone model):

1. **Drivers.** `WindowsHostPlatform` installs WinUSB automatically through `usb-driver-installer` and shows one UAC prompt. To do it by hand with **Zadig**:
   - Make sure step 2 is done first, then run the app once so the phone switches to AOA. The app's open step fails at this point, which is expected.
   - In Zadig choose **Options → List All Devices**, pick `18D1:2D00` (or `18D1:2D01` **Interface 0**), select **WinUSB** and click **Install Driver**. The device usually appears as "Android Accessory Interface".
   - Run the app again.
2. **Phone in normal mode.** `switchDeviceToAoa` needs to open the phone before it switches. That works if **USB debugging is on** and the Google/OEM ADB driver is installed (that interface is WinUSB-based). Otherwise install WinUSB for the phone's own VID:PID; this hides the phone from Explorer/MTP while installed.
3. **adb.** Close Android Studio's device monitor, or run `adb kill-server`. `WindowsHostPlatform.prepare()` does this automatically.

Verify:

```
# Device Manager → "Universal Serial Bus devices" should list the accessory with driver "WinUSB"
pnputil /enum-devices /connected | findstr /i "VID_18D1"
```

Windows-specific troubleshooting:

Symptom	Fix
`open → -12 ERROR_NOT_SUPPORTED` on `18D1:2D00/2D01`	WinUSB not installed, or Plug and Play binding hasn't finished. Retries handle the binding delay; otherwise install the driver
`open → -3 ERROR_ACCESS`	`adb.exe` or another tool holds the device. Kill `adb`, close OEM suites (Samsung Smart Switch, etc.)
Phone reappears as `2D01` and open fails	USB debugging on gives the composite PID. WinUSB must be on **MI_00** (interface 0). Or turn USB debugging off to get `2D00`
AOA switch returns `false`	No WinUSB-compatible interface on the phone in normal mode. Turn USB debugging on, or allow the driver install on the phone VID:PID
Installer returns `UnsignedDriverBlocked`	Corporate policy blocks the driver. Ask IT to pre-install the WinUSB package
Works once, fails after replug	A different USB port can mean a new device instance on some PCs. Run the install again for that instance

3.8 Linux setup

One-time per PC. Create `/etc/udev/rules.d/51-secureqt.rules`:

```
# Google AOA accessory (18D1:2D00-2D05) and Google devices
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0660", GROUP="plugdev", TAG+="uaccess"
# Your phones in normal mode: add one line per vendor (examples: Samsung, Xiaomi, OnePlus, Motorola)
SUBSYSTEM=="usb", ATTR{idVendor}=="04e8", MODE="0660", GROUP="plugdev", TAG+="uaccess"
SUBSYSTEM=="usb", ATTR{idVendor}=="2717", MODE="0660", GROUP="plugdev", TAG+="uaccess"
SUBSYSTEM=="usb", ATTR{idVendor}=="2a70", MODE="0660", GROUP="plugdev", TAG+="uaccess"
SUBSYSTEM=="usb", ATTR{idVendor}=="22b8", MODE="0660", GROUP="plugdev", TAG+="uaccess"
```

Then:

```
sudo groupadd -f plugdev
sudo usermod -aG plugdev "$USER"          # log out and back in for the group to apply
sudo udevadm control --reload-rules && sudo udevadm trigger
# unplug and replug the phone
```

`TAG+="uaccess"` gives the logged-in desktop user access without the group. `GROUP="plugdev"` covers SSH and headless sessions.

Verify:

```
lsusb | grep -i 18d1                      # after connect: "18d1:2d00 Google Inc." (or 2d01)
ls -l /dev/bus/usb/$(lsusb -d 18d1: | awk '{print $2"/"substr($4,1,3)}')
                                          # should show group plugdev and rw for the group, or a '+' ACL
```

Linux-specific troubleshooting:

Symptom	Fix
`open → -3 ERROR_ACCESS`	udev rule missing, or the vendor ID isn't listed. Add it, reload, replug. Running once with `sudo` confirms it's a permission problem
`claimInterface → -6 ERROR_BUSY`	A kernel driver or another process has the interface. `setAutoDetachKernelDriver` handles the kernel case; check `lsof /dev/bus/usb/BBB/DDD` for processes
AOA switch works, then the device keeps disconnecting	ModemManager probing the phone. Add `ENV{ID_MM_DEVICE_IGNORE}="1"` to the rule lines, or `sudo systemctl stop ModemManager`
Phone vanishes from the file manager while connected	Expected: in AOA mode the phone stops exposing MTP until it leaves accessory mode (unplug)

Symptom	Fix
`UnsatisfiedLinkError: libusb4java`	Your CPU architecture has no bundled native in usb4java 1.3.0. Use an x86-64 JDK, or build `libusb4java` for your architecture

3.9 Shipping the host app

	Windows	Linux
Package	`jpackage --type msi` / `exe` (or Compose Desktop `packageMsi`)	`jpackage --type deb` / `rpm` (or `packageDeb`)
Install-time step	Optional: pre-install WinUSB for `18D1:2D00` and `18D1:2D01 MI_00` from the installer so users never see a UAC prompt	Install `51-secureqt.rules` to `/etc/udev/rules.d/` in the package's post-install script and run `udevadm control --reload-rules && udevadm trigger`
Runtime privileges	Normal user	Normal user (member of `plugdev` or logged-in seat)

---

4. Android (AAR) integration

4.1 Add dependencies

```
// app/build.gradle.kts
dependencies {
    implementation(files("libs/secureqt-sdk-android.aar"))
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.10.2")
}
```

Requirements: `minSdk 24`, Java 11 compile options.

4.2 Accessory filter

The `manufacturer` / `model` here **must match** what the desktop passes to `switchDeviceToAoa()`.

`res/xml/accessory_filter.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <usb-accessory
        manufacturer="AndroidOpenAccessory"
        model="DataTransfer"
        version="1.0" />
</resources>
```

4.3 Manifest

```
<manifest ...>
    <uses-feature android:name="android.hardware.usb.accessory" android:required="true" />

    <application ...>
        <activity android:name=".MainActivity" android:exported="true" android:launchMode="singleTask">
            <intent-filter>
                <action android:name="android.hardware.usb.action.USB_ACCESSORY_ATTACHED" />
            </intent-filter>
            <meta-data
                android:name="android.hardware.usb.action.USB_ACCESSORY_ATTACHED"
                android:resource="@xml/accessory_filter" />
        </activity>
```

```
    </application>
</manifest>
```

When the activity is launched by `USB_ACCESSORY_ATTACHED`, the user's permission is granted implicitly. If you open the accessory any other way, call `UsbManager.requestPermission(accessory, pendingIntent)` first.

4.4 Open the transport & handshake

```
import android.hardware.usb.UsbAccessory
import android.hardware.usb.UsbManager
import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.createAndroidAccessoryTransport

val usbManager = getSystemService(USB_SERVICE) as UsbManager
val accessory: UsbAccessory = intent.getParcelableExtra(UsbManager.EXTRA_ACCESSORY)
    ?: usbManager.accessoryList?.firstOrNull()
    ?: return

lifecycleScope.launch(Dispatchers.IO) {
    val transport = SecureQtSdk.createAndroidAccessoryTransport(usbManager, accessory)
        ?: return@launch   // null → no permission or accessory already open

    val channel = SecureQtSdk.createChannel(transport)
    if (!channel.performHandshake(isInitiator = false)) {   // Android is the responder
        transport.close(); return@launch
    }

    runCommandLoop(channel)   // §4.5
    transport.close()
}
```

> Run the session in a **foreground service** for long transfers so Android doesn't kill it when the screen turns off.

4.5 Implement the command loop

The Android side must answer each command in the exact format the desktop client expects (see §5). Minimal example:

```
import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.channel.SecureChannel
import com.example.secureqt.sdk.protocol.CommandPayloadParser
import com.example.secureqt.sdk.protocol.StandardCommandIds
import java.io.File
import java.io.FileInputStream
import java.io.FileOutputStream
import java.nio.ByteBuffer

suspend fun runCommandLoop(channel: SecureChannel) {
    while (true) {
        val raw = try {
            channel.receiveSecure() ?: break           // null → disconnected
        } catch (e: SecureChannel.TransferCancelledException) {
            continue                                   // host cancelled; keep listening
        }
        val cmd = CommandPayloadParser.parse(raw) ?: continue

        when (cmd.commandId) {
            StandardCommandIds.CMD_LIST_DIR -> {
                val dir = File(CommandPayloadParser.readString(cmd.payload))
                val items = dir.listFiles().orEmpty()
                channel.sendSecure(ByteBuffer.allocate(4).putInt(items.size).array())
                for (f in items) {
                    val name = f.name.toByteArray(Charsets.UTF_8)
                    channel.sendSecure(
```

```
                        ByteBuffer.allocate(1 + 8 + 4 + name.size)
                            .put(if (f.isDirectory) 1.toByte() else 0.toByte())
                            .putLong(if (f.isFile) f.length() else 0L)
                            .putInt(name.size).put(name)
                            .array()
                    )
                }
            }

            StandardCommandIds.CMD_SEND_FILE_START -> {
                val (path, size) = CommandPayloadParser.readFileStart(cmd.payload)
                val out = File(path).apply { parentFile?.mkdirs() }
                FileOutputStream(out).use { fos ->
                    var received = 0L
                    while (true) {
                        val (type, body) = channel.receiveRawPacket() ?: break
                        if (type == SecureQtSdk.Packets.TYPE_CANCEL) { out.delete(); break }
                        if (type != SecureQtSdk.Packets.TYPE_DATA) break        // TYPE_EOF
                        val chunk = channel.decryptData(body) ?: break
                        fos.write(chunk); received += chunk.size
                    }
                }
            }

            StandardCommandIds.CMD_FETCH_FILE_START -> {
                val f = File(CommandPayloadParser.readString(cmd.payload))
                val size = if (f.isFile) f.length() else 0L
                channel.sendSecure(ByteBuffer.allocate(8).putLong(size).array())
                if (f.isFile) FileInputStream(f).use { input ->
                    val buf = ByteArray(256 * 1024)
                    while (true) {
                        val n = input.read(buf); if (n == -1) break
                        channel.sendSecure(buf.copyOf(n))
                    }
                }
                channel.sendRawPacket(SecureQtSdk.Packets.TYPE_EOF, ByteArray(0))
            }

            StandardCommandIds.CMD_DELETE_FILE -> {
                val ok = File(CommandPayloadParser.readString(cmd.payload)).deleteRecursively()
                channel.sendSecure(byteArrayOf(if (ok) 1.toByte() else 0.toByte()))
            }

            StandardCommandIds.CMD_RENAME_FILE -> {
                val (oldPath, newName) = CommandPayloadParser.readTwoStrings(cmd.payload)
                val src = File(oldPath)
                val ok = src.renameTo(File(src.parentFile, newName))
                channel.sendSecure(byteArrayOf(if (ok) 1.toByte() else 0.toByte()))
            }

            StandardCommandIds.CMD_CREATE_FOLDER -> {
                val ok = File(CommandPayloadParser.readString(cmd.payload)).mkdirs()
                channel.sendSecure(byteArrayOf(if (ok) 1.toByte() else 0.toByte()))
            }

            StandardCommandIds.CMD_SET_DEVICE_ID -> {
                val id = CommandPayloadParser.readString(cmd.payload)   // no reply expected
            }

            StandardCommandIds.CMD_DISCONNECT -> break

            // CMD_SEND_DIR_ZIP_START / CMD_FETCH_DIR_ZIP_START: same as file send/fetch,
            // but the stream is a ZIP (unzip on receive / zip on send).
        }
    }
    channel.disconnect()
}
```

For large files, use a read → decrypt → write pipeline with coroutine `Channel`s; `SecureTransferClientImpl` does this on the host side. `encryptAndWrapData()` + `sendPrebuiltPacket()` let you encrypt on `Dispatchers.Default` and send on `Dispatchers.IO` in parallel.

---

5. Standard commands & wire contract

Every command is sent as **one encrypted `TYPE_DATA` packet**: `[1: commandId][args]`. All integers are **big-endian**; strings are `[4: byteLen][UTF-8 bytes]`.

ID	Constant	Host sends (after cmd byte)	Android must reply
`0x00`	`CMD_LIST_DIR`	`str path`	secure `[4: count]`, then `count` × secure `[1: isDir][8: size][4: nameLen][name]`
`0x01`	`CMD_SEND_FILE_START`	`str fullPath` `[8: size]`, then `TYPE_DATA` … then `TYPE_EOF` (or `TYPE_CANCEL`)	nothing
`0x02`	`CMD_FETCH_FILE_START`	`str path`	secure `[8: size]` (0 if missing), then `TYPE_DATA` …, then raw `TYPE_EOF`
`0x03`	`CMD_FETCH_DIR_ZIP_START`	`str path`	secure `[8: size]`, then ZIP stream as `TYPE_DATA` …, then `TYPE_EOF`
`0x04`	`CMD_DISCONNECT`	–	nothing (close session)
`0x05`	`CMD_SEND_DIR_ZIP_START`	`str fullPath` `[8: 0]`, then ZIP stream, then `TYPE_EOF`	nothing
`0x06`	`CMD_DELETE_FILE`	`str path`	secure `[1: 0x01 ok / 0x00 fail]`
`0x07`	`CMD_RENAME_FILE`	`str oldPath` `str newName`	secure `[1: ok]`
`0x08`	`CMD_CREATE_FOLDER`	`str path`	secure `[1: ok]`
`0x09`	`CMD_SET_DEVICE_ID`	`str deviceId`	nothing

Outer packet types (`SecureQtSdk.Packets`):

Byte	Name	Encrypted?	Meaning
`0x01`	`TYPE_PUBLIC_KEY`	No	ECDH public key (handshake)
`0x02`	`TYPE_DATA`	**Yes** (`[12: IV][ciphertext+16: GCM tag]`)	Command / response / file chunk
`0x03`	`TYPE_EOF`	No	End of stream
`0x04`	`TYPE_PING`	No	Keep-alive (ignored by `receiveSecure`)
`0x05`	`TYPE_ERROR`	No	Error
`0x06`	`TYPE_CANCEL`	No	Transfer aborted

Frame layout: `[1: type][4: payloadLength][payload]` (`SecureQtSdk.Codec.HEADER_SIZE == 5`).

---

6. Custom commands

Use IDs that don't collide with the standard set (for example `0x10` and up). Add new commands without changing the SDK:

Host (JAR):

```
const val CMD_GET_APP_VERSION: Byte = 0x10
const val CMD_ECHO: Byte = 0x11

// No arguments
val reply: ByteArray? = client.executeCustomCommand(CMD_GET_APP_VERSION)
val version = reply?.toString(Charsets.UTF_8)

// With arguments: pass ONLY the arguments; executeCustomCommand prepends commandId itself
val text = "hello".toByteArray()
val args = java.nio.ByteBuffer.allocate(4 + text.size).putInt(text.size).put(text).array()
val echo = client.executeCustomCommand(CMD_ECHO, args)
```

> ⚠️ Don't pass the output of `CommandPayloadBuilder.build*()` to `executeCustomCommand`. Those builders already include the command byte, so the ID would be sent twice. Use the builders with `channel.sendSecure(...)` directly instead.

Android (AAR): reply with **exactly one** `sendSecure()`, because `executeCustomCommand` reads a single response:

```
when (cmd.commandId) {
    0x10.toByte() -> channel.sendSecure(BuildConfig.VERSION_NAME.toByteArray())
    0x11.toByte() -> channel.sendSecure(CommandPayloadParser.readString(cmd.payload).toByteArray())
}
```

Encoding helpers (`com.example.secureqt.sdk.protocol.CommandPayloadBuilder`, not exposed on the `SecureQtSdk` facade, so import it directly):

Builder	Wire format	Matching parser (`CommandPayloadParser`)
`buildSimple(cmd)`	`[cmd]`	`parse()`
`buildWithString(cmd, s)`	`[cmd][4][s]`	`readString()`
`buildFileStart(cmd, path, size)`	`[cmd][4][path][8]`	`readFileStart()`
`buildWithTwoStrings(cmd, a, b)`	`[cmd][4][a][4][b]`	`readTwoStrings()`
`buildCustom(cmd, vararg segs)`	`[cmd]([4][seg])*`	`readString()` / manual

Other readers: `readInt()`, `readLong()`, `readBoolean()`.

---

7. API reference

`SecureQtSdk` (common)

Member	Description
`VERSION`	`"1.0.0"`
`createChannel(transport): SecureChannel`	Wrap any `IUsbTransport` in an encrypted channel
`createTransferClient(channel): ISecureTransferClient`	High-level host client
`createBuffer()`	Sliding window buffer (used internally)
`Packets` / `Codec`	Packet type constants / frame encoder–decoder
`CommandPayloadParser`	Command decoder
`Utils`	`FormatUtils`

JVM-only extensions (`import com.example.secureqt.sdk.*`)

Function	Description
`SecureQtSdk.switchDeviceToAoa(device, manufacturer, model): Boolean`	Sends AOA requests 51/52/53; the device then re-enumerates
`SecureQtSdk.findDesktopBulkEndpoints(device): BulkEndpoints?`	`interfaceNumber`, `inAddr`, `outAddr`
`SecureQtSdk.createDesktopTransport(handle, inEp, outEp, ifaceNum)`	`DesktopAoaTransport`; call `close()` when done

Android-only extensions

Function	Description
`SecureQtSdk.createAndroidAccessoryTransport(usbManager, accessory)`	`AndroidAoaTransport?`; `null` if it can't be opened. Call `close()` when done

`SecureChannel`

Method	Description
`suspend performHandshake(isInitiator): Boolean`	ECDH exchange. Waits up to ~30 s; the initiator resends its key every 1 s
`suspend sendSecure(bytes): Boolean`	Encrypt + send one `TYPE_DATA` packet
`suspend receiveSecure(): ByteArray?`	Next decrypted `TYPE_DATA`; skips EOF/PING/unknown; throws `TransferCancelledException` on `TYPE_CANCEL`; `null` on disconnect
`suspend sendRawPacket(type, payload)` / `receiveRawPacket(timeoutMs)`	Unencrypted framed I/O (control packets, streaming)
`encryptAndWrapData(bytes)` + `sendPrebuiltPacket(packet)`	Split encrypt/send for parallel pipelines
`decryptData(payload)`	Decrypt a raw `TYPE_DATA` payload
`disconnect()`	Wipe session keys and buffers

`ISecureTransferClient` (host)

`listRemoteFiles`, `sendFile`, `sendDirectory`, `fetchFile`, `fetchRemoteDirectory`, `deleteFile`, `renameFile`, `createFolder`, `sendDeviceId`, `sendDisconnect`, `cancelTransfer`, `executeCustomCommand`. All except `cancelTransfer` are `suspend`. Progress callbacks are `(bytesDone: Long, total: Long) -> Unit`, throttled to about every 250 ms. For directory uploads `total` is `0` (unknown).

Custom transport

Implement `IUsbTransport` to run the protocol over any byte pipe (for example a loopback for tests):

```
interface IUsbTransport {
    suspend fun read(buffer: ByteArray): Int      // bytes read, 0 = nothing yet, -1 = disconnected
    suspend fun write(buffer: ByteArray): Boolean
    fun isConnected(): Boolean
    fun clearBuffer()
}
```

8. Limits, threading & gotchas

- **Max packet payload: 10 MB.** Larger frames are rejected. The client streams files in 256 KB chunks.
- **One command at a time.** `SecureTransferClientImpl` serialises calls with a mutex, so concurrent calls queue up.
- **Call from coroutines.** All I/O is `suspend` and switches to `Dispatchers.IO` internally. Don't call it from the Android main thread with `runBlocking`.
- **The host must claim the USB interface.** `createDesktopTransport` doesn't call `LibUsb.claimInterface`.
- **The device re-enumerates after `switchDeviceToAoa`.** The original `Device` object becomes invalid, so scan again for VID `0x18D1`.
- **Filter strings must match.** If `manufacturer`/`model` differ from `accessory_filter.xml`, the app won't launch automatically.
- **Responses must match the contract in §5 exactly.** A missing reply leaves the host waiting in `receiveSecure()`.
- **Keys are ephemeral.** A new ECDH key pair is created for every `performHandshake`, and there is no persistent pairing or identity check. Treat the physical cable as the trust boundary.

---

9. Troubleshooting

Symptom	Likely cause / fix
`switchDeviceToAoa` returns `false`	No permission to open the device (Linux udev / Windows driver), or the device doesn't support AOA
`LibUsb.open` → `ERROR_ACCESS (-3)` on Windows	`adb` server holds the device. Run `adb kill-server`
`LibUsb.open` → `ERROR_NOT_SUPPORTED (-12)`	WinUSB driver missing for `18D1:2D00/2D01`. Run the driver installer, then retry after about 1–2 s
`claimInterface` → `ERROR_BUSY`	Another process or kernel driver owns it. Detach the kernel driver or close the other app
`performHandshake` returns `false` after ~30 s	Android app didn't open the accessory or isn't running the handshake as responder
`createAndroidAccessoryTransport` returns `null`	No USB permission, or the accessory is already opened by another component
App doesn't open on plug-in	`accessory_filter.xml` strings don't match `switchDeviceToAoa` arguments
Transfer stalls on Android	Write sizes are already split into 16 KB bursts by `AndroidAoaTransport`. Check that the receiver loop is consuming packets
`listRemoteFiles` returns empty	Android didn't reply with the `[4: count]` header first, or has no storage permission

---

Project layout

```
secureqt-sdk/src
├── commonMain/…/sdk
│   ├── SecureQtSdk.kt              Public facade
```

```
│   ├── channel/SecureChannel.kt    Encrypted, framed channel
│   ├── client/                     ISecureTransferClient + impl (host operations)
│   ├── crypto/                     ECDH P-256 + AES-256-GCM
│   ├── protocol/                   PacketCodec, PacketTypes, StandardCommandIds, Builder/Parser
│   ├── buffer/ model/ util/ logging/
│   └── transport/IUsbTransport.kt
├── jvmMain/…/sdk                   DesktopAoaTransport, DesktopAoaConnector, desktop extensions
└── androidMain/…/sdk               AndroidAoaTransport, Android extensions
```

See also, in the repo root: `SECUREQT_SDK_PROTOCOL_SPECIFICATION.md`, `Protocol.md`, `Architecture_and_Security.md`.